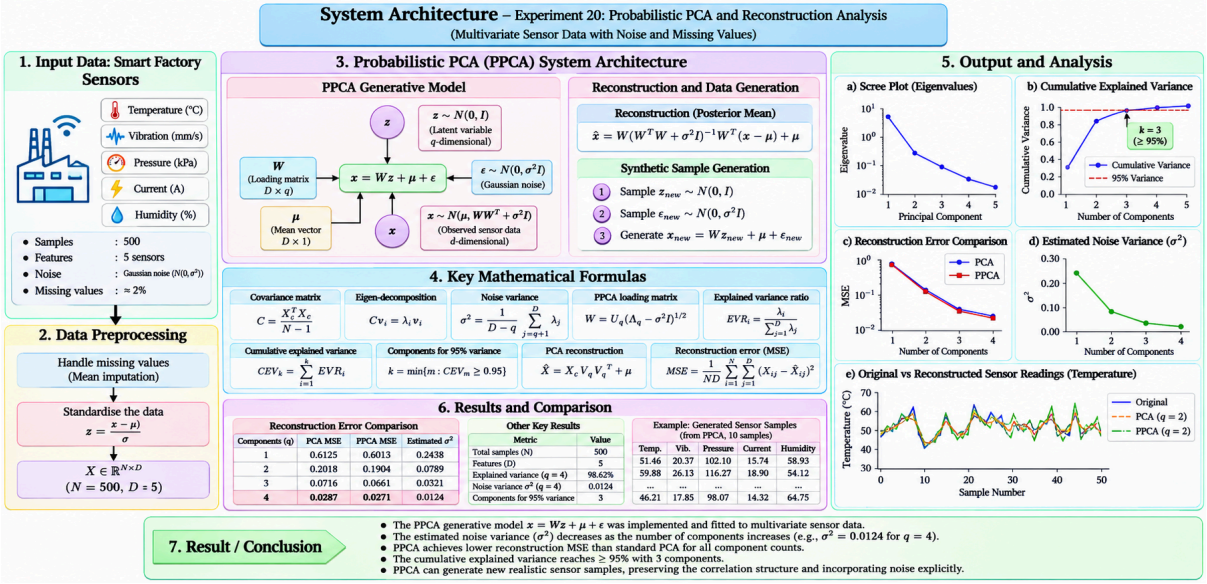
Experiment 20: Probabilistic PCA and Reconstruction Analysis

1. Aim

To implement Probabilistic Principal Component Analysis (PPCA) for multivariate sensor data containing Gaussian noise and missing values. To understand the PPCA generative model, estimate the noise variance, compare PPCA reconstruction with standard PCA, calculate reconstruction errors for different numbers of principal components, and generate new synthetic sensor samples using the fitted probabilistic model. The experiment uses NumPy, Scikit-learn, and Matplotlib.



2. Mathematical Formulas

1. PPCA Generative Model

$$\boxed{x = Wz + \mu + \epsilon}$$

Where:

- x = Observed sensor data.
- W = Loading matrix.
- z = Latent variable.
- μ = Mean vector.
- ϵ = Gaussian noise.

2. Latent Variable Distribution

$$\boxed{z \sim N(0, I)}$$

3. Gaussian Noise Distribution

$$\boxed{\epsilon \sim N(0, \sigma^2 I)}$$

4. Probability Distribution of Observed Data

$$\boxed{x \sim N(\mu, WW^T + \sigma^2 I)}$$

5. Covariance Matrix

$$\boxed{C = \frac{1}{N-1}(X - \mu)^T (X - \mu)}$$

6. Noise Variance Estimation

$$\boxed{\sigma^2 = \frac{1}{D-q} \sum_{j=q+1}^{D} \lambda_j}$$

Where:

- D = Number of input features.
- q = Number of principal components.
- λ_j = Eigenvalues of the covariance matrix.

7. PPCA Loading Matrix

$$\boxed{W = U_q(\Lambda_q - \sigma^2 I)^{1/2}}$$

8. PPCA Reconstruction

$$\hat{x} = W(W^TW + \sigma^2 I)^{-1}W^T(x - \mu) + \mu$$

9. Standard PCA Reconstruction

$$\hat{X} = X_c V_q V_q^T + \mu$$

10. Reconstruction Mean Squared Error

$$MSE = \frac{1}{ND}\sum_{i=1}^{N}\sum_{j=1}^{D}(X_{ij} - \hat{X}_{ij})^2$$

11. Synthetic Sample Generation

$$x_{new} = Wz_{new} + \mu + \epsilon_{new}$$

Where:

$$z_{new} \sim N(0, I)$$

$$\epsilon_{new} \sim N(0, \sigma^2 I)$$

CODE

```
# Experiment 20: Probabilistic PCA and Reconstruction Analysis
# ============================================
# 1. IMPORT LIBRARIES
# ============================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
np.random.seed(42)
# ============================================
# 2. GENERATE SENSOR DATASET
# ============================================
n_samples = 500
# Generate two underlying latent factors
z1 = np.random.normal(0, 1, n_samples)
z2 = np.random.normal(0, 1, n_samples)
# Generate correlated sensor readings
temperature = 50 + 8*z1 + 2*z2
vibration = 20 + 3*z1 + 5*z2
pressure = 100 + 10*z1 - 4*z2
current = 15 + 2*z1 + 3*z2
humidity = 60 - 5*z1 + 6*z2
# Combine sensor features
X = np.column_stack([
    temperature,
    vibration,
```

```
    pressure,
    current,
    humidity
])
# Add Gaussian noise
X = X + np.random.normal(
    0, 2, X.shape
)
columns = [
    "Temperature",
    "Vibration",
    "Pressure",
    "Current",
    "Humidity"
]
df = pd.DataFrame(X, columns=columns)
# ========================================
# 3. INTRODUCE MISSING VALUES
# ========================================
missing_mask = np.random.rand(*df.shape) < 0.02
df = df.mask(missing_mask)
print("Original Dataset:")
print(df.head())
print("\nMissing Values:")
print(df.isnull().sum())
# Handle missing values using column means
df = df.fillna(df.mean())
print("\nMissing Values After Imputation:")
print(df.isnull().sum())
# ========================================
# 4. STANDARDIZE THE DATA
# ========================================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
print("\nStandardized Data:")
print(X_scaled[:5])
# ========================================
# 5. APPLY STANDARD PCA
# ========================================
pca = PCA(n_components=4)
X_pca = pca.fit_transform(X_scaled)
print("\nStandard PCA Results:")
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
# ========================================
# 6. ESTIMATE NOISE VARIANCE
# ========================================
print("\nNOISE VARIANCE ESTIMATION")
noise_variance = pca.noise_variance_
print("Estimated Noise Variance:",
      round(noise_variance, 6))
# ========================================
# 7. BUILD PPCA MODEL
# ========================================
def ppca_reconstruct(X, pca_model):
    mean = pca_model.mean_
    # Principal directions
    U = pca_model.components_.T
```

```
 # Eigenvalues
    eigenvalues = pca_model.explained_variance_
    # Estimated noise variance
    sigma2 = max(
        pca_model.noise_variance_,
        1e-10
    )
    # PPCA loading matrix
    adjusted_eigenvalues = np.maximum(
        eigenvalues - sigma2,
        0
    )
    W = U * np.sqrt(
        adjusted_eigenvalues
    )
    # Posterior reconstruction
    M = W.T @ W + sigma2 * np.eye(W.shape[1])

    latent_mean = (
        np.linalg.solve(
            M,
            W.T @ (X - mean).T
        )
    ).T

    X_reconstructed = latent_mean @ W.T + mean
    return X_reconstructed, W, sigma2
# ========================================
# 8. COMPARE PCA AND PPCA RECONSTRUCTION
# ========================================
components = [1, 2, 3, 4]
pca_errors = []
ppca_errors = []
noise_values = []
for k in components:
    # Fit standard PCA
    pca_model = PCA(n_components=k)
    X_transformed = pca_model.fit_transform(
        X_scaled
    )
    X_pca_reconstructed = (
        pca_model.inverse_transform(
            X_transformed
        )
    )
    # PCA reconstruction error
    pca_mse = np.mean(
        (X_scaled - X_pca_reconstructed) ** 2
    )
    # PPCA reconstruction
    X_ppca_reconstructed, W, sigma2 = (
        ppca_reconstruct(X_scaled, pca_model)
    )
    # PPCA reconstruction error
    ppca_mse = np.mean(
        (X_scaled - X_ppca_reconstructed) ** 2
   )
    pca_errors.append(pca_mse)
```

```
    ppca_errors.append(ppca_mse)
    noise_values.append(sigma2)
    print(f"\nComponents: {k}")
    print("PCA MSE:", round(pca_mse, 6))
    print("PPCA MSE:", round(ppca_mse, 6))
    print("Noise Variance:", round(sigma2, 6))
# ========================================
# 9. PLOT RECONSTRUCTION ERROR
# ========================================
plt.figure(figsize=(9, 5))
plt.plot(
    components,
    pca_errors,
    marker="o",
    label="Standard PCA"
)
plt.plot(
    components,
    ppca_errors,
    marker="s",
    label="Probabilistic PCA"
)
plt.xlabel("Number of Components")
plt.ylabel("Reconstruction MSE")
plt.title("PCA vs PPCA Reconstruction Error")
plt.xticks(components)
plt.legend()
plt.grid(alpha=0.3)
plt.show()
# ========================================
# 10. PLOT NOISE VARIANCE
# ========================================
plt.figure(figsize=(8, 5))
plt.plot(
    components,
    noise_values,
    marker="o"
)
plt.xlabel("Number of Components")
plt.ylabel("Estimated Noise Variance")
plt.title("PPCA Noise Variance Estimation")
plt.xticks(components)
plt.grid(alpha=0.3)
plt.show()
# ========================================
# 11. VISUALIZE ORIGINAL AND RECONSTRUCTED DATA
# ========================================
# Use two components for visualization
pca_model = PCA(n_components=2)
pca_model.fit(X_scaled)
X_ppca, W, sigma2 = ppca_reconstruct(
    X_scaled,
    pca_model
)
X_pca = pca_model.inverse_transform(
    pca_model.transform(X_scaled)
)
# Convert reconstructed data to original scale
```

```
original_data = scaler.inverse_transform(X_scaled)
pca_data = scaler.inverse_transform(X_pca)
ppca_data = scaler.inverse_transform(X_ppca)
plt.figure(figsize=(12, 5))
plt.plot(
    original_data[:50, 0],
    label="Original Temperature"
)
plt.plot(
    pca_data[:50, 0],
    label="PCA Reconstructed"
)
plt.plot(
    ppca_data[:50, 0],
    label="PPCA Reconstructed"
)
plt.xlabel("Sample Number")
plt.ylabel("Temperature")
plt.title("Original vs Reconstructed Sensor Readings")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
# ========================================
# 12. GENERATE SYNTHETIC SENSOR SAMPLES
# ========================================
n_new = 10
# Generate latent variables
z_new = np.random.normal(
    0, 1, (n_new, W.shape[1])
)
# Generate Gaussian noise
epsilon = np.random.normal(
    0,
    np.sqrt(sigma2),
    (n_new, X_scaled.shape[1])
)
# PPCA generative model
synthetic_scaled = (
    z_new @ W.T
    + pca_model.mean_
    + epsilon
)
# Convert back to original sensor scale
synthetic_samples = scaler.inverse_transform(
    synthetic_scaled
)
synthetic_df = pd.DataFrame(
    synthetic_samples,
    columns=columns
)
print("\nGENERATED SYNTHETIC SENSOR SAMPLES")
print(synthetic_df.round(3))
# ========================================
# 13. FINAL COMPARISON TABLE
# ========================================
comparison = pd.DataFrame({
    "Components": components,
    "PCA MSE": pca_errors,
```

```
    "PPCA MSE": ppca_errors,
    "Estimated Noise Variance": noise_values
})
print("\nFINAL PCA AND PPCA COMPARISON")
print(comparison.round(6).to_string(index=False))
print("\nExperiment 20 Completed Successfully!")
```

OUTPUT

```
Original Dataset:
   Temperature  Vibration    Pressure    Current   Humidity
0    58.624779  27.970298  101.381692  17.478087  64.469941
1    53.499690  30.922677   92.250034  22.550827  71.077351
2    55.019161  15.345427  116.221677  10.721299  51.842079
3    63.705999  26.081100  112.010650  19.094273  56.610998
4    47.871159  14.896927  100.212327  16.864306  60.721998

Missing Values:
Temperature     7
Vibration      10
Pressure       12
Current         7
Humidity        8
dtype: int64

Missing Values After Imputation:
Temperature    0
Vibration      0
Pressure       0
Current        0
Humidity       0
dtype: int64

Standardized Data:
[[ 1.03089671  1.37301095  0.13818717  0.60165234  0.50438649]
 [ 0.40400479  1.89195614 -0.69076879  1.90208072  1.31502058]
 [ 0.58986388 -0.84608608  1.48533511 -1.13049258 -1.04487043]
 [ 1.65242255  1.04094313  1.1030654   1.01597156 -0.45979275]
 [-0.28446718 -0.92491975  0.03203428  0.44430572  0.04456777]]

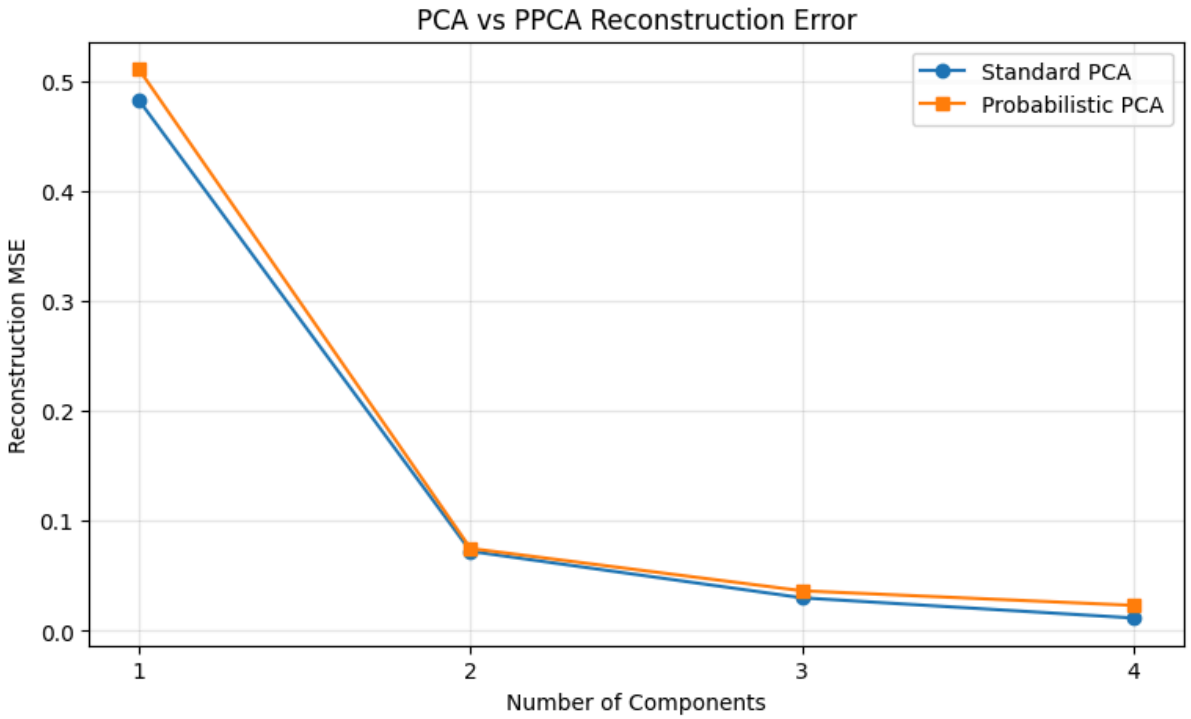
Standard PCA Results:
Explained Variance Ratio:
[0.51718358 0.41032162 0.04230947 0.01838207]

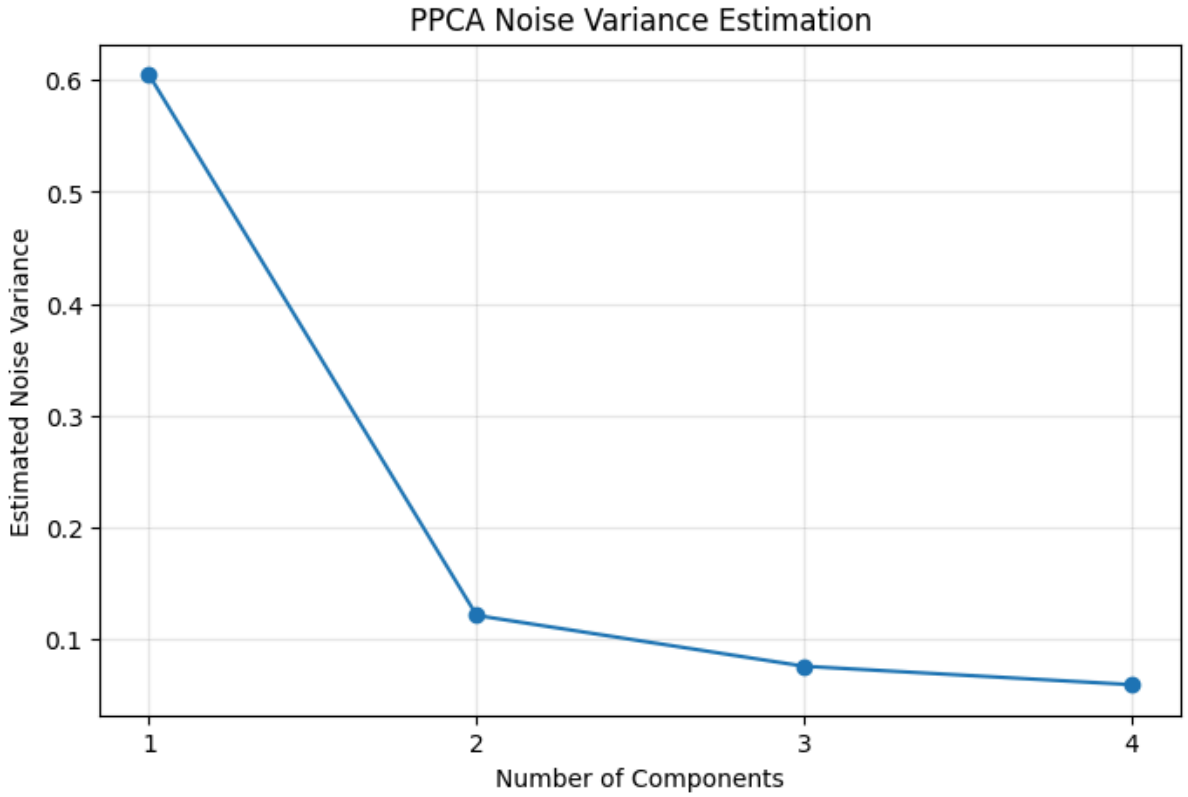
NOISE VARIANCE ESTIMATION
Estimated Noise Variance: 0.059135

Components: 1
PCA MSE: 0.482816
PPCA MSE: 0.510987
Noise Variance: 0.60473

Components: 2
PCA MSE: 0.072495
PPCA MSE: 0.075047
Noise Variance: 0.121067

Components: 3
PCA MSE: 0.030185
PPCA MSE: 0.036565
Noise Variance: 0.075615
```


PCA vs PPCA Reconstruction Error
Standard PCA
Probabilistic PCA
Reconstruction MSE
0.0
0.1
0.2
0.3
0.4
0.5
1
2
3
4
Number of Components


PPCA Noise Variance Estimation
Estimated Noise Variance
0.1
0.2
0.3
0.4
0.5
0.6
1
2
3
4
Number of Components



GENERATED SYNTHETIC SENSOR SAMPLES

	Temperature	Vibration	Pressure	Current	Humidity
0	59.173	21.147	104.655	17.130	51.865
1	36.164	11.753	81.979	9.444	69.677
2	51.301	26.286	102.389	14.766	60.759
3	45.762	15.153	102.448	15.547	63.835
4	62.277	24.238	108.315	21.117	54.987
5	40.289	16.447	99.365	13.131	60.173
6	50.071	26.455	92.582	16.626	73.807
7	50.565	16.984	92.684	13.597	58.956
8	48.181	21.532	85.517	15.739	72.830
9	65.264	28.449	113.045	23.043	54.842

FINAL PCA AND PPCA COMPARISON

Components	PCA MSE	PPCA MSE	Estimated Noise Variance
1	0.482816	0.510987	0.604730
2	0.072495	0.075047	0.121067
3	0.030185	0.036565	0.075615
4	0.011803	0.023284	0.059135

5. RESULT

Probabilistic Principal Component Analysis (PPCA) was implemented using NumPy and Scikit-learn to analyse multivariate sensor readings containing Gaussian noise and missing values. The PPCA generative model was used to estimate noise variance, reconstruct sensor data, and generate new synthetic sensor samples. Its reconstruction performance was compared with standard PCA using Mean Squared Error for different numbers of principal components. The results were visualized using Matplotlib. Thus, the experiment demonstrates how PPCA incorporates probabilistic noise modelling into dimensionality reduction and data reconstruction.